

04. Vite

Detailed beginner handbook - English and Arabic

Technologies: Vite

What you will learn

Run a development server and create a production build.

Vite serves source files quickly in development and bundles an optimised dist folder for release.

الهدف: بناء مثال صغير وفهمه خطوة بخطوة.

شرح بسيط: تعلم الفكرة أولاً، ثم جرب المثال الصغير ببيانات اصطناعية فقط.

How to study

Study one lesson at a time. Type examples yourself, predict the result, run the code, then change one thing. Keep a notebook of new words, errors, root causes, and fixes.

Course map

Setup and mental model; ten core concepts; a guided mini-project; the real dashboard architecture; debugging; exercises and answers.

Lesson 1 - Setup and mental model

Prerequisites

Windows 10 or 11, VS Code, PowerShell, and permission to install learning dependencies. Use only synthetic data in tutorials and screenshots.

Setup sequence

1. Open PowerShell.
2. Go to learning-examples/04-vite.
3. Read README.md.
4. Run `npm install`; `npm run dev`; `npm run build`.
5. Read the complete error if it fails.
6. Change one safe value and rerun.

الخطوات: افتح مجلد المثال، شغل الأمر، غير قيمة واحدة، ولاحظ النتيجة.

Mental model

Treat Vite as one layer in a system. Identify the input, the transformation or rule, the output, and possible failures. Understanding that flow is more valuable than memorising syntax.

Checkpoint

Explain: What problem does this technology solve? What is its input? What output should it produce? What can fail?

Lesson 2 - What a development server and bundler do

What is it?

A development server serves modules quickly and supports live updates. A production bundler resolves dependencies, transforms syntax, optimises assets, and writes deployable files.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In Vite, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
npm create vite@latest learning-app
cd learning-app
npm install
npm run dev
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates What a development server and bundler do.

What a development server and bundler do. اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك. هذا الدرس يشرح: do

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 3 - Creating a project and understanding index.html as the entry point

What is it?

Vite treats index.html as part of the module graph. Its module script imports the application entry file, which then imports components, styles, and assets.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In Vite, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
<!-- index.html -->
<div id="root"></div>
<script type="module" src="/src/main.js"></script>
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Creating a project and understanding index.html as the entry point.

Creating a project and understanding index.html as the entry point: هذا الدرس يشرح: اكتب المثال بنفسك، حدد المدخلات والعمليات والنتيجة، ثم غير قيمة واحدة لتفهم السلوك.

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 4 - ES modules, dependency resolution, and hot module replacement

What is it?

Native ES modules use import and export. Vite pre-bundles dependencies and replaces changed modules through hot module replacement without refreshing the entire page.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In Vite, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
// math.js
export const add = (a,b) => a+b;
// main.js
import {add} from './math.js';
console.log(add(2,3));
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates ES modules, dependency resolution, and hot module replacement.

هذا الدرس يشرح: ES modules, dependency resolution, and hot module replacement. اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك.

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 5 - npm scripts and development versus production modes

What is it?

npm run dev starts development mode; npm run build creates dist; npm run preview serves that build locally. Production behaviour must be checked separately from development.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In Vite, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
{
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  }
}
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates npm scripts and development versus production modes.

هذا الدرس يشرح: npm scripts and development versus production modes. اكتب المثال بنفسك، حدد المدخلات والعمليات والنتيجة، ثم غير قيمة واحدة لتفهم السلوك.

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 6 - Environment variables and the VITE_ public prefix

What is it?

Only variables prefixed VITE_ are exposed to browser code through import.meta.env. Never place passwords, private keys, or server secrets in frontend environment variables.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In Vite, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
// .env
VITE_API_URL=http://127.0.0.1:8000
// source
console.log(import.meta.env.VITE_API_URL);
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Environment variables and the VITE_ public prefix.

Environment variables and the VITE_ public prefix: هذا الدرس يشرح: اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك.

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 7 - Static assets, public files, imports, and URLs

What is it?

Imported assets receive processed URLs and hashes. Files in public are copied unchanged and referenced from the root. Choose imports when the asset belongs to source code.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In Vite, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
import imageUrl from './logo.png';
const image = document.createElement('img');
image.src = imageUrl;
// public/manual.pdf is referenced as /manual.pdf
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Static assets, public files, imports, and URLs.

Static assets, public files, imports, and URLs. اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لفهم السلوك.
هذا الدرس يشرح: URLs

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 8 - TypeScript and React plugin integration

What is it?

Plugins extend Vite's transforms. The React plugin handles JSX and fast refresh; TypeScript is transpiled by Vite while tsc should run separately for type checking.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In Vite, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
// vite.config.js
import {defineConfig} from 'vite';
import react from '@vitejs/plugin-react';
export default defineConfig({plugins:[react()]});
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates TypeScript and React plugin integration.

TypeScript and React plugin integration. اكتب المثال بنفسك، حدد المدخلات والعمليات والنتيجة، ثم غير قيمة واحدة لتفهم السلوك.
هذا الدرس يشرح:

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 9 - Development proxy configuration for a local API

What is it?

A development proxy forwards selected paths such as /api to a backend server, avoiding hard-coded origins and simplifying local CORS behaviour.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In Vite, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
export default defineConfig({
  server: { proxy: { '/api': 'http://127.0.0.1:8000' } }
});
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Development proxy configuration for a local API.

Development proxy configuration for a local API. اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك. هذا الدرس يشرح:

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 10 - Production builds, chunks, source maps, and preview

What is it?

Build output may split code into chunks loaded on demand. Source maps connect built errors to source. preview verifies asset paths and production-only problems.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In Vite, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
npm run build
# Inspect dist/
npm run preview
# Test the production files locally
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Production builds, chunks, source maps, and preview.

Production builds, chunks, source maps, and preview: هذا الدرس يشرح: اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لفهم السلوك.

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 11 - Diagnosing missing modules, stale caches, and build failures

What is it?

Read the first build error, confirm the import path and filename case, inspect package versions, clear only relevant caches, and reproduce with the smallest module.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In Vite, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
npm ls vite
npm run build
# Read the first error, then check its file and import path.
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Diagnosing missing modules, stale caches, and build failures.

Diagnosing missing modules, stale caches, and build failures: اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك. هذا الدرس يشرح:

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 7 - Guided mini-project

Project goal

Run a development server and create a production build.

Starting example

```
{  
  "scripts": { "dev": "vite", "build": "vite build" }  
}
```

Read the code

Find imported tools or page elements. Identify the synthetic input. Find the function, event, route, or transformation that changes it. Finally, locate where the result is displayed, returned, saved, or packaged.

Build sequence

1. Run the untouched example.
2. Record the result.
3. Rename one unclear value.
4. Add a synthetic record.
5. Validate empty input.
6. Validate incorrect input.
7. Add a helpful error.
8. Rerun every case.
9. Compare with exercise-solution.
10. Explain every line.

Definition of done

Normal, empty, and invalid cases behave deliberately; no private data is used; and you can explain every line without reading the guide.

Lesson 8 - The real dashboard

Architecture connection

The application combines Vite with the rest of the stack. The frontend gathers filters and files; the local API validates requests; the data layer transforms workbook rows; charts present aggregates; export tools create files; and the desktop layer packages the application for Windows.

Trace the upload feature

The user selects a workbook. The frontend sends it to the local API. The backend validates the file and sheets. The data layer creates safe tables. The API returns metadata. React updates state. Plotly displays results. Identify exactly where this guide's technology participates.

Privacy and quality

Do not place narrative, contact, or identifying fields in tutorials, logs, screenshots, tests, or exports. Use generated identifiers and synthetic totals. Validate at each boundary and collect only fields required for the calculation.

Architecture exercise

Draw five boxes: UI, API, Data Processing, Export, Desktop Packaging. Add arrows and label the data. Circle the box related to this guide and add two validation checks.

Lesson 9 - Debugging

Reliable debugging loop

1. Reproduce with the smallest input.
2. Read the first meaningful error.
3. Find the exact file and line.
4. Inspect value and type.
5. Compare expected and actual results.
6. Change one thing.
7. Rerun the same case.
8. Add a regression test.

Common beginner mistakes

- Wrong folder or command.
- Missing dependency or inactive environment.
- Misspelled file, variable, field, route, or import.
- Assuming input is always present.
- Mixing setup, processing, and display in one block.
- Hiding an exception rather than understanding it.
- Using real sensitive data in a test.

أخطاء شائعة: تشغيل الأمر من مجلد خاطئ، نسيان تثبيت التبعيات، أو تغيير أشياء كثيرة بمرة واحدة.

Error notebook

Record: command, expected result, actual error, root cause, smallest fix, and test added. This turns errors into reusable knowledge.

Lesson 10 - Exercises and answers

Exercises

1. Explain the technology in three sentences.
2. Recreate the example without copying.
3. Add one normal synthetic record.
4. Handle empty input.
5. Handle invalid input.
6. Improve unclear names.
7. Add or describe one test.
8. Locate the technology in the dashboard.
9. Record one error and root cause.
10. Add one mini-project feature.

تمرين: أضف شهرا أو قيمة أمانة جديدة، ثم اشرح ما حدث بكلماتك.

Answer guide

A strong solution is observable and explainable: the documented command works; output changes for the new record; empty and invalid cases have deliberate results; names communicate purpose; a test states an expected result; and the architecture explanation identifies the correct boundary.

Next step

Next: 05. Python Foundations